

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación

FIGURAS GEOMÉTRICAS 2D

Aplicación de escritorio con C# y Windows Forms

NRC: 29505

Asignatura: Computación Gráfica

Docente:

Ing. Morales Caiza Darío Javier

Integrantes:

Josue Andel Chnago Parra

Simbaña Rivadeneira Michael Steven

Link:

<https://github.com/Josue-Chango/Trabajo_{poligonos}.git>

Fecha: 29 de abril del 2026

1. Introducción

Este trabajo consiste en una aplicación de escritorio desarrollada en C# con Windows Forms, orientada a dibujar figuras geométricas bidimensionales a partir de parámetros ingresados por el usuario.

La arquitectura del sistema está basada en programación orientada a objetos:

- **Figuras:** maneja el dibujo con GDI+
- **Calculos:** contiene fórmulas matemáticas

El sistema incluye figuras básicas, polígonos regulares y formas especiales, con escalado automático para visualización.

2. Objetivos

2.1. Objetivo General

Desarrollar una aplicación de escritorio en C# con Windows Forms que permita al usuario ingresar parámetros de distintas figuras geométricas bidimensionales, visualizarlas dinámicamente en un panel de dibujo y calcular sus propiedades matemáticas (área y perímetro).

2.2. Objetivos Específicos

- Implementar métodos de dibujo con GDI+
- Calcular área y perímetro
- Aplicar escalado automático
- Cubrir múltiples figuras geométricas

3. Desarrollo

La aplicación se organiza en dos clases principales: Figuras, que maneja el renderizado GDI+, y Calculos, que provee las fórmulas matemáticas. Ambas viven en el namespace `shappes2dysoninstanci`.

Todos los métodos de dibujo reciben un objeto Graphics como primer parámetro, que representa la superficie del PanelGrafico. Las figuras se anclan en coordenadas fijas (generalmente alrededor de $x=350$, $y=100$ o $y=150$) para garantizar que aparezcan centradas y sin recortes. El antialiasing (`SmoothingMode.AntiAlias`) se aplica en todas las figuras que usan polígonos para evitar el efecto pixelado en los bordes diagonales.

3.1. Funcionalidades Implementadas

3.1.1. Funcionalidades Mínimas

- **Menú de navegación:** La aplicación tiene un menú principal (`FrmHome`) desde donde se abre el formulario independiente de cada figura. No hay que recordar atajos ni rutas complejas.
- **Posicionamiento controlado:** Las figuras se dibujan en coordenadas fijas dentro del `PanelGrafico` (principalmente $x = 350$), evitando que se corten o salgan del área visible.
- **Colores diferenciados:** Cada figura utiliza colores específicos:
 - Polígonos regulares: relleno multicolor por secciones triangulares
 - Figuras básicas: `LightBlue` con borde azul
 - Cometa: púrpura suave
 - Rombo: `Thistle` con borde morado
 - Flecha: amarillo con borde negro
- **Escalado automático:** Si los valores ingresados son menores a 10 (o menores a 15 en el triángulo escaleno), se multiplican por un factor (10 o 4 según la figura), garantizando visibilidad sin necesidad de manejar píxeles.

3.1.2. Requerimientos No Funcionales

- **Código organizado:** Cada figura posee su propio formulario. La lógica de dibujo y cálculo están separadas en `Figuras.cs` y `Calculos.cs`.
- **Herramientas de dibujo adecuadas:** Se emplean:
 - `Pen` para contornos
 - `SolidBrush` / `Brushes` para rellenos
 - `PointF` para precisión en vértices
 - `Graphics` como contexto de dibujo
- **Interfaz clara:** Cada formulario incluye campos específicos según la figura. La validación de entrada evita errores, como violaciones de la desigualdad triangular.
- **Separación de responsabilidades:** `Figuras.cs` se encarga del dibujo, `Calculos.cs` de los cálculos, y los formularios gestionan la interacción con el usuario.

3.2. Figuras Implementadas

A continuación se detalla cada figura con sus variables, lógica de coordenadas, colores GDI+, métodos de dibujo y fórmulas matemáticas.

3.2.1. Círculo (Circle)

Se trabaja con el radio como valor base. Los métodos DrawEllipse y FillEllipse esperan un rectángulo delimitador, por lo que la posición se ajusta restando el radio al punto de anclaje para que el círculo quede centrado en (350, 100). Cuando el radio ingresado es menor a 10, se multiplica por 10 antes de dibujar.

Variables	radio — radio del círculo en cm
Lógica / Coordenadas	$\text{if (radio} < 10) \rightarrow r = \text{radio} \times 10$ Pos. X = $350 - r$ Pos. Y = $100 - r$ Ancho = $2 \times r$ Alto = $2 \times r$
Colores (GDI+)	Relleno: LightBlue. Borde: Color Blue, 2px.
Métodos GDI+	DrawEllipse(Pen azul 2px, ...) FillEllipse(Brushes.LightBlue, ...)
Área y Perímetro	Área: $\pi \cdot \text{radio}^2$ Perímetro: $2 \cdot \pi \cdot \text{radio}$

3.2.2. Elipse (Ellipse)

La elipse utiliza dos radios independientes: radioMayor (eje horizontal) y radioMenor (eje vertical). Al igual que el círculo, el rectángulo delimitador se posiciona restando cada radio al punto de anclaje. Ambos valores se escalan por 10 si son menores a 10.

Variables	radioMayor — semieje horizontal radioMenor — semieje vertical
Lógica / Coordenadas	if (radioMayor <10 && radioMenor <10) → $r_m = r_M = 10$ Pos. X = $350 - r_m$ Pos. Y = $100 - r_M$ Ancho = $2 \times r_m$ Alto = $2 \times r_M$
Colores (GDI+)	Relleno: LightBlue . Borde: Color Blue , 2px.
Métodos GDI+	DrawEllipse(Pen azul 2px, ...) FillEllipse(Brushes.LightBlue, ...)
Área y Perímetro	Área: $\pi \cdot \text{radioMayor} \cdot \text{radioMenor}$ Perímetro (Aprox. Ramanujan): $P \approx \pi \left[3(a+b) - \sqrt{(3a+b)(a+3b)} \right]$

3.2.3. Ovalo

Comparte exactamente la misma implementación que la elipse: dos radios independientes y el mismo cálculo de rectángulo delimitador. Se dibuja con los mismos métodos y la misma lógica de escalado. La distinción con la elipse es semántica en el contexto del formulario que lo llama.

Variables	radioMayor — semieje horizontal radioMenor — semieje vertical
Lógica / Coordenadas	if (radioMayor <10 && radioMenor <10) $\rightarrow \times 10$ Pos. X = $350 - rm$ Pos. Y = $100 - rM$ Ancho = $2 \times rm$ Alto = $2 \times rM$
Colores (GDI+)	Relleno: LightBlue . Borde: Color Blue , 2px.
Métodos GDI+	DrawEllipse(Pen azul 2px, ...) FillEllipse(Brushes.LightBlue, ...)
Área y Perímetro	Área: $\pi \cdot \text{radioMayor} \cdot \text{radioMenor}$

3.2.4. Cuadrado (Square)

Un solo parámetro: el lado. Se dibuja como un rectángulo con ancho y alto iguales. Si el lado es menor a 10, se multiplica por 10. El punto de anclaje es (350, 100).

Variables	lado — longitud del lado en cm
Lógica / Coordenadas	if (lado <10) $\rightarrow l = \text{lado} \times 10$ DrawRectangle desde (350, 100) con ancho = l , alto = l
Colores (GDI+)	Relleno: LightBlue . Borde: Color Blue , 2px.
Métodos GDI+	DrawRectangle(Pen azul 2px, ...) FillRectangle(Brushes.LightBlue, ...)
Área y Perímetro	Área: lado^2 Perímetro: $4 \times \text{lado}$

3.2.5. Rectángulo (Rectangle)

Posee dos parámetros: `weight` (ancho) y `height` (alto). Si ambos son menores a 6, se multiplican por 4 antes de dibujar; de lo contrario, se usan directamente. El punto de anclaje es (350, 100).

Variables	<code>weight</code> — ancho en cm <code>height</code> — alto en cm
Lógica / Coordenadas	<code>if (weight < 6 && height < 6) → × 4</code> <code>DrawRectangle</code> desde (350, 100) con ancho = <code>weight</code> , alto = <code>height</code>
Colores (GDI+)	Borde: Pens.Red (sin relleno explícito)
Métodos GDI+	<code>DrawRectangle(Pens.Red, ...)</code>
Área y Perímetro	Área: $\text{weight} \times \text{height}$ Perímetro: $2 \times (\text{weight} + \text{height})$

3.2.6. Triángulo Escaleno

Recibe los tres lados y verifica primero la **desigualdad triangular**: si cualquier lado supera la suma de los otros dos, el método retorna sin dibujar. El tercer vértice se calcula con la Ley de los Cosenos. Si algún lado es menor a 15, todos se escalan por 10.

Variables	ladoA, ladoB, ladoC — tres lados distintos
Lógica / Coordenadas	Validación: $A + B > C, A + C > B, B + C > A$ (si falla $\rightarrow$ return) if (a < 15) $\rightarrow$ a, b, c = $\times$ 10 $p1 = (350, 200)$ $p2 = (350 + c, 200)$ $cx = (b^2 + c^2 - a^2) / (2c)$ $cy = \sqrt{b^2 - cx^2}$ $p3 = (350 + cx, 200 - cy)$
Colores (GDI+)	Relleno: LightBlue . Borde: Color Blue , 2px
Métodos GDI+	FillPolygon(Brushes.LightBlue, ...) DrawPolygon(Pen azul 2px, ...)
Área y Perímetro	Perímetro: $ladoA + ladoB + ladoC$ Área (Fórmula de Herón): $\sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$ donde $s = \frac{a+b+c}{2}$

3.2.7. Triángulo Equilátero

Un solo parámetro: el lado. El vértice superior se centra en (350, 150) y los dos inferiores se calculan con la fórmula de altura del triángulo equilátero. Se escala por 10 si el lado es menor a 10.

Variables	lado — longitud de cada lado en cm
Lógica / Coordenadas	if (lado < 10) $\rightarrow l = \text{lado} \times 10$ altura = $\left(\frac{\sqrt{3}}{2}\right) \times l$ superior: (350, 150 – altura/2) inf. derecha: (350 + l/2, 150 + altura/2) inf. izquierda: (350 – l/2, 150 + altura/2)
Colores (GDI+)	Relleno: LightBlue . Borde: Color Blue , 2px.
Métodos GDI+	FillPolygon(Brushes.LightBlue, ...) DrawPolygon(Pen azul 2px, ...)
Área y Perímetro	Perímetro: $\text{ladoA} + \text{ladoB} + \text{ladoC}$ (los tres iguales) Área: $\sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$ (Fórmula de Herón)

3.2.8. Triángulo Rectángulo

Posee dos parámetros: base y altura. La esquina del ángulo recto queda en (350, 200). Los otros vértices se extienden a lo largo de la base horizontal y hacia arriba sobre la vertical. Se escala por 10 si ambos son menores a 10.

Variables	<code>baseTri</code> — longitud de la base <code>alturaTri</code> — longitud de la altura
Lógica / Coordenadas	<code>if (base <10 && altura <10) → × 10</code> $p1 = (350, 200)$ ← ángulo recto $p2 = (350 + b, 200)$ ← extremo de la base $p3 = (350, 200 - h)$ ← extremo de la altura
Colores (GDI+)	Relleno: LightBlue . Borde: Color Blue , 2px.
Métodos GDI+	<code>FillPolygon(Brushes.LightBlue, ...)</code> <code>DrawPolygon(Pen azul 2px, ...)</code>
Área y Perímetro	Perímetro: $ladoA + ladoB + ladoC$ (con hipotenusa = $\sqrt{base^2 + altura^2}$) Área: $\sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$ (Fórmula de Herón)

3.2.9. Paralelogramo

Recibe dos lados y el ángulo de inclinación en grados. El desplazamiento horizontal y vertical del lado inclinado se obtiene con las funciones *seno* y *coseno*. Si ambos lados son menores a 10, se escalan por 10. El anclaje se sitúa en (350, 150).

Variables	lado — longitud de cada lado anguloGrados — ángulo agudo del rombo (°)
Lógica / Coordenadas	if (lado < 10) $\rightarrow l = \text{lado} \times 10$ radianes = $(\text{anguloGrados}/2) \times (\pi/180)$ $\text{semiDiagH} = l \cdot \cos(\text{radianes})$ $\text{semiDiagV} = l \cdot \sin(\text{radianes})$ superior: (350, 150 – semiDiagV) derecha: (350 + semiDiagH, 150) inferior: (350, 150 + semiDiagV) izquierda: (350 – semiDiagH, 150)
Colores (GDI+)	Relleno: Thistle . Borde: Color.Purple , 2px.
Métodos GDI+	FillPolygon(Brushes.Thistle, ...) DrawPolygon(Pen morado 2px, ...)

3.2.11. Trapecio

Recibe base mayor, base menor, lado izquierdo y lado derecho. Si alguno es menor a 10, todos se escalan por 10. La altura se calcula geométricamente usando la diferencia entre bases y los lados. La diferencia total determina el desplazamiento de los vértices superiores.

Variables	lado — radio del pentágono inscrito
Lógica / Coordenadas	<pre> if (lado < 10) → a = lado × 4 incremento = 2π/5 para i = 0..4: angle = i × 2π/5 x = 350 + a × cos(angle) y = 100 + a × sin(angle) Relleno por triángulos: centro + puntos[i] + puntos[i+1] </pre>
Colores (GDI+)	Relleno: Red, Green, Blue, Yellow, Orange (por sección). Borde: Black.
Métodos GDI+	FillPolygon(brushes[i], triángulo) DrawPolygon(Pens.Black, puntos)
Área y Perímetro	Área: $\frac{\text{lado}^2 \times \sqrt{25+10\sqrt{5}}}{4}$ Perímetro: $5 \times \text{lado}$

3.2.13. Hexágono

Polígono regular de 6 lados. Mismo principio que el pentágono pero dividiendo el círculo en 6 partes. La técnica de relleno por triángulos con colores distintos hace que cada sección del hexágono tenga su propio color. Escalado por 4 si el lado es menor a 10.

Variables	lado — radio del hexágono inscrito
Lógica / Coordenadas	if (lado < 10) $\rightarrow a = \text{lado} \times 4$ $\text{incremento} = \pi/3 \quad (= 2\pi/6)$ [cite: 1] para $i = 0,5$: $\text{angle} = i \times \pi/3$ [cite: 1] $x = 350 + a \times \cos(\text{angle})$ $y = 100 + a \times \sin(\text{angle})$ Relleno por triángulos: centro + puntos[i] + puntos[i+1]
Colores (GDI+)	Relleno: Red, Green, Blue, Yellow, Orange, Purple (por sección).
Métodos GDI+	FillPolygon(brushes[i], triángulo) (sin DrawPolygon explícito en este método)
Área y Perímetro	Área: $\frac{3 \times \sqrt{3} \times \text{radio}^2}{2}$ [cite: 1] Perímetro: $6 \times \text{radio}$

3.2.14. Heptágono

Polígono regular de 7 lados. Utiliza la misma técnica trigonométrica y de relleno por triángulos que el pentágono y hexágono, pero con 7 secciones. El heptágono no tiene construcción geométrica exacta con regla y compás, por lo que la fórmula paramétrica es el único camino directo.

Variables	lado — radio del heptágono inscrito
Lógica / Coordenadas	<pre> if (lado < 10) → a = lado × 4 incremento = 2π/7[cite: 1] para i = 0..6: angle = i × 2π/7 x = 350 + a × cos(angle) y = 100 + a × sin(angle) Relleno: centro + puntos[i] + puntos[(i+1) % 7] </pre>
Colores (GDI+)	Relleno: Red, Green, Blue, Yellow, Orange, Purple, Cyan[cite: 1] Borde: Black[cite: 1]
Métodos GDI+	<pre> FillPolygon(brushes[i], triángulo) DrawPolygon(Pens.Black, puntos) </pre>
Área y Perímetro	Área: $\frac{7 \times \text{lado}^2}{4 \times \tan(\pi/7)}$ (lado = $2 \times \text{radio} \times \sin(\pi/7)$) Perímetro: $7 \times \text{lado}$ (donde lado = $2 \times \text{radio} \times \sin(\pi/7)$)

3.2.15. Octágono

Polígono regular de 8 lados. Los vértices están separados 45° entre sí. Sigue el mismo patrón de construcción trigonométrica y relleno multicolor por triángulos que las figuras anteriores.

Variables	lado — radio del octágono inscrito
Lógica / Coordenadas	if (lado < 10) $\rightarrow a = \text{lado} \times 4$ $\text{incremento} = 2\pi/8 \quad (= \pi/4 = 45^\circ)$ para $i = 0,7$: $\text{angle} = i \times 2\pi/8$ $x = 350 + a \times \cos(\text{angle})$ $y = 100 + a \times \sin(\text{angle})$ Relleno: centro + puntos[i] + puntos[(i+1)%8]
Colores (GDI+)	Relleno: Red, Green, Blue, Yellow, Orange, Purple, Cyan, Magenta Borde: Black
Métodos GDI+	FillPolygon(brushes[i], triángulo) DrawPolygon(Pens.Black, puntos)
Área y Perímetro	Área: $\frac{2 \times \text{lado}^2}{\tan(\pi/8)}$ (lado = $2 \times \text{radio} \times \sin(\pi/8)$) Perímetro: $8 \times \text{lado}$ (donde lado = $2 \times \text{radio} \times \sin(\pi/8)$)

3.2.16. Nonágono

Polígono regular de 9 lados. Los vértices se separan 40° entre sí. A partir de aquí los polígonos regulares se parecen bastante a un círculo.

Variables	lado — radio del nonágono inscrito
Lógica / Coordenadas	if (lado < 10) $\rightarrow a = \text{lado} \times 4$ $\text{incremento} = 2\pi/9 \quad (\approx 40^\circ)$ para $i = 0, 8$: $\text{angle} = i \times 2\pi/9$ $x = 350 + a \times \cos(\text{angle})$ $y = 100 + a \times \sin(\text{angle})$ Relleno: centro + puntos[i] + puntos[(i+1) % 9]
Colores (GDI+)	Relleno: Red, Green, Blue, Yellow, Orange, Purple, Cyan, Magenta, Brown Borde: Black
Métodos GDI+	FillPolygon(brushes[i], triángulo) DrawPolygon(Pens.Black, puntos)
Área y Perímetro	Área: $\frac{9 \times \text{lado}^2}{4 \times \tan(\pi/9)}$ (lado = $2 \times \text{radio} \times \sin(\pi/9)$) Perímetro: $9 \times \text{lado}$ (donde lado = $2 \times \text{radio} \times \sin(\pi/9)$)

3.2.17. Decágono

Polígono regular de 10 lados. Presenta una separación de 36° entre vértices. La lógica de construcción es la misma que la de los anteriores polígonos regulares.

Variables	lado — radio del decágono inscrito
Lógica / Coordenadas	if (lado < 10) $\rightarrow a = \text{lado} \times 4$ $\text{incremento} = 2\pi/10 \quad (= \pi/5 = 36^\circ)$ para $i = 0,9$: $\text{angle} = i \times 2\pi/10$ $x = 350 + a \times \cos(\text{angle})$ $y = 100 + a \times \sin(\text{angle})$ Relleno: centro + puntos[i] + puntos[(i+1)%10]
Colores (GDI+)	Relleno: Red, Green, Blue, Yellow, Orange, Purple, Cyan,[cite: 1] Magenta, Brown, OldLace. Borde: Black
Métodos GDI+	FillPolygon(brushes[i], triángulo) DrawPolygon(Pens.Black, puntos)
Área y Perímetro	Área: $\frac{5 \times \text{lado}^2}{4 \times \tan(\pi/10)}$ (lado = $2 \times \text{radio} \times \sin(\pi/10)$) Perímetro: $10 \times \text{lado}$ (donde lado = $2 \times \text{radio} \times \sin(\pi/10)$)

3.2.18. Cometa

La cometa recibe ancho y alto. El punto superior está al 25 % del alto desde el centro ($pSup = \text{alto} \times 0,25f$), lo que le otorga su forma asimétrica característica: la parte superior es más corta que la inferior. Si el ancho es menor a 10, ambos se escalan por 10.

Variables	ancho — ancho de la cometa alto — alto total de la cometa
Lógica / Coordenadas	if (ancho < 10) $\rightarrow w, h = \times 10$ $pSup = h \times 0,25f$ (25 % del alto hacia arriba) superior: $(350, 150 - pSup)$ derecha: $(350 + w/2, 150)$ inferior: $(350, 150 + (h - pSup))$ izquierda: $(350 - w/2, 150)$
Colores (GDI+)	Relleno: RGB(188, 153, 203)[cite: 1] Borde: RGB(60, 60, 90), 2px
Métodos GDI+	FillPolygon(SolidBrush púrpura claro, ...) DrawPolygon(Pen oscuro 2px, ...)

3.2.19. Estrella

Se dibuja una estrella de 5 puntas utilizando la técnica de radios concéntricos. Se calculan 10 puntos alternando entre un radio exterior (puntas) y un radio interior (valles), distribuidos uniformemente cada 36° . El escalado multiplica el radio por 10 si es menor a 10.

Variables	<code>radioExterior</code> — distancia del centro a las puntas en cm
Lógica / Coordenadas	<pre> if (radioExterior <10) → rExt = radioExterior × 10 rInt = rExt × 0.4 para i = 0,9: ángulo = -90° + (i × 36°) x = 350 + radio × cos(ángulo) y = 150 + radio × sin(ángulo) (radio alterna entre rExt y rInt) </pre>
Colores (GDI+)	Relleno: Yellow . Borde: Color.Orange , 2px
Métodos GDI+	<pre> FillPolygon(Brushes.Yellow, puntos) DrawPolygon(Pen naranja 2px, puntos) </pre>
Área y Perímetro	No implementados en la clase <code>Calculos</code> para esta figura

3.2.20. Corazón

Se implementa mediante curvas de Bézier para lograr una forma fluida. Se utiliza un `GraphicsPath` que une dos curvas simétricas desde una hendidura superior hasta una punta inferior, controladas por puntos de tensión laterales.

Variables	tamaño — factor de escala para el corazón en cm
Lógica / Coordenadas	<pre> if (tamaño <10) → t = tamaño × 10 topCenter = (350, 150 - t × 0,3) bottomPoint = (350, 150 + t × 0,7) Curvas Bézier con puntos de control a (centerX ± t) </pre>
Colores (GDI+)	Relleno: Red . Borde: Color.DarkRed , 2px
Métodos GDI+	<pre> FillPath(Brushes.Red, path) DrawPath(Pen rojo oscuro 2px, path) </pre>
Área y Perímetro	No implementados en la clase <code>Calculos</code> para esta figura

3.2.21. Luna

Representa una fase lunar creciente generada por la intersección de dos arcos. Se dibuja un arco exterior de 180° y se cierra la figura con un arco interior de sentido opuesto, desplazado lateralmente para crear el efecto de “mordida”.

Variables	radio — radio del círculo exterior en cm
Lógica / Coordenadas	$\text{if } (\text{radio} < 10) \rightarrow r = \text{radio} \times 10$ $\text{rectExterior} = (350 - r, 150 - r, 2r, 2r)$ $\text{rectInterior} = (350 - r \cdot 0,6, 150 - r, 2r, 2r)$ Arcos: exterior (90° a 270°), interior (270° a 90° en reversa)
Colores (GDI+)	Relleno: RGB(190, 215, 110) Borde: RGB(45, 50, 75), 3px
Métodos GDI+	FillPath(SolidBrush verde claro, path) DrawPath(Pen oscuro 3px, path)
Área y Perímetro	No implementados en la clase Calculos para esta figura

3.2.22. Cruz

Figura ortogonal de 12 vértices que forman una cruz griega. El diseño es proporcional al tamaño del brazo ingresado, extendiéndose simétricamente desde el centro (350, 150).

Variables	tamañoBrazo — grosor y largo proporcional en cm
Lógica / Coordenadas	if (tamañoBrazo < 10) $\rightarrow t = \text{tamañoBrazo} \times 10$ Anclaje central: (350, 150) Cálculo de 12 puntos: coordinados por $t/2$ y $t \times 1,5$
Colores (GDI+)	Relleno: RGB(40, 160, 220) Borde: RGB(40, 50, 70), 3px
Métodos GDI+	FillPolygon(SolidBrush celeste, puntos) DrawPolygon(Pen oscuro 3px, puntos)
Área y Perímetro	No implementados en la clase Calculos para esta figura

3.2.23. Pie

Dibuja una sección de un círculo definida por un ángulo de barrido. Utiliza un rectángulo delimitador convertido a enteros para compatibilidad con GDI+. El barrido dinámico permite crear diversas formas de “rebanada”.

Variables	radio — radio del sector en cm grados — ángulo de barrido (°)
Lógica / Coordenadas	if (radio <10) $\rightarrow r = \text{radio} \times 10$ rect = (int)(350-r, 150-r, 2r, 2r) DrawPie desde 0° con barrido de 'grados'
Colores (GDI+)	Relleno: RGB(180, 160, 130) Borde: RGB(60, 55, 80), 3px
Métodos GDI+	FillPie(SolidBrush café, rect, 0, grados) DrawPie(Pen oscuro 3px, rect, 0, grados)
Área y Perímetro	No implementados en la clase Calculos para esta figura

3.2.24. Flecha

La flecha se construye con 7 puntos que forman el cuerpo rectangular y la punta triangular. Las mitades de altura_cuerpo y altura_cabeza centran la figura verticalmente respecto al punto de anclaje (350,150). No tiene escalado automático.

Variables	base_cuerpo — largo del cuerpo altura_cuerpo — alto del cuerpo base_cabeza — largo de la punta altura_cabeza — alto total de la punta
Lógica / Coordenadas	$\text{mitad_cuerpo} = \text{altura_cuerpo} / 2$ $\text{mitad_cabeza} = \text{altura_cabeza} / 2$ $p1 = (350, 150 - \text{mitad_cuerpo}) \quad \leftarrow \text{esquina sup. izq.}$ $p2 = (350 + b_c, 150 - \text{mitad_cuerpo}) \quad \leftarrow \text{esquina sup. der.}$ $p3 = (350 + b_c, 150 - \text{mitad_cabeza}) \quad \leftarrow \text{expansión cabeza}$ $p4 = (350 + b_c + b_h, 150) \quad \leftarrow \text{punta}$ $p5 = (350 + b_c, 150 + \text{mitad_cabeza}) \quad \leftarrow \text{contracción cabeza}$ $p6 = (350 + b_c, 150 + \text{mitad_cuerpo}) \quad \leftarrow \text{esquina inf. der.}$ $p7 = (350, 150 + \text{mitad_cuerpo}) \quad \leftarrow \text{esquina inf. izq.}$
Colores (GDI+)	Relleno: Yellow . Borde: Black
Métodos GDI+	<code>FillPolygon(Brushes.Yellow, ...)</code> <code>DrawPolygon(Pens.Black, ...)</code>
Área y Perímetro	Área: $(\text{base_cuerpo} \times \text{altura_cuerpo}) + \frac{\text{base_cabeza} \times \text{altura_cabeza}}{2}$ Perímetro: $2 \times (b_c + h_c) + b_h + 2 \times \sqrt{(\frac{b_h}{2})^2 + (h_h)^2} - h_c$ (Nota: h_h corresponde a la altura de la cabeza)

4. Conclusiones

El proyecto logró lo que se planteó: una aplicación funcional que dibuja 18 figuras geométricas distintas y calcula sus propiedades matemáticas. La separación entre Figuras.cs y Calculos.cs resultó una decisión acertada, ya que mantuvo el código organizado y fácil de seguir incluso cuando el catálogo de figuras creció.

El escalado automático resolvió bien el problema de usabilidad: el usuario ingresa medidas en centímetros sin preocuparse por píxeles, y el sistema se encarga de que todo sea visible. No es un enfoque perfecto (un escalado adaptativo al tamaño del panel sería más robusto), pero funciona de manera consistente.

Trabajar con los polígonos regulares fue más directo de lo esperado: una vez que se entiende el patrón trigonométrico, escalar de pentágono a decágono es solo cambiar el divisor. Lo más interesante fue la variedad de enfoques: primitivas rectangulares para cuadrado y rectángulo,

ley de cosenos para triángulos escalenos, ecuación paramétrica para el corazón, y 7 puntos específicos para la flecha. En conjunto, el sistema cubre bastante terreno para ser un trabajo introductorio de computación gráfica.

5. Recomendaciones

- El hexágono no llama DrawPolygon, así que el contorno no se dibuja. Sería bueno unificar ese comportamiento con el resto de polígonos.
- El escalado fijo ($\times 10$ o $\times 4$) podría reemplazarse por un cálculo dinámico basado en el tamaño real del PanelGrafico, lo que haría la interfaz más adaptable a distintas resoluciones.
- Agregar las fórmulas de área y perímetro del rombo, paralelogramo y cometa completaría la clase Calculos, ya que esas figuras actualmente no tienen cálculos implementados.
- Para el corazón y otras figuras curvas, usar GraphicsPath con curvas Bézier mejoraría la calidad visual, especialmente en tamaños pequeños donde el polígono se ve anguloso.
- Exponer opciones de color y grosor de borde desde el formulario haría la aplicación bastante más flexible sin cambiar nada de la lógica de dibujo.

6. Link

https://github.com/Josue-Chango/Trabajo_poligonos.git